

Service Level Objectives & Canarying Releases

Reliability you can spend. How to put a number on "good enough", turn the remainder into a budget, and buy information with it instead of losing it to accidents.

1 Why 100% is wrong The target below perfection, and the budget that falls out of it.	2 What to measure Good events over total events; specification versus implementation; starter numbers.	3 Making it bite Agreement, policy, and deciding with the budget instead of arguing.	4 Spending it to learn Canarying: population, duration, metrics, and what isn't a canary.
--	---	---	--

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

THE RULE THAT DOES MOST OF THE WORK

If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

1 — Learning goal and roadmap	orientation
2 — Why 100% is the wrong target	teaching · questions · matrix
3 — What to measure, and how to get a first number	teaching · questions · matrix
4 — Making it bite: agreement, policy, decisions	teaching · questions · matrix
5 — Spending the budget to buy information	teaching · questions · matrix
6 — Concept sketch + master matrix	consolidation
7 — Symptom → diagnosis → move	working reference
8 — Numbers worth remembering · Glossary	reference
9 — Answer key	keep covered
10 — Spaced-review tracker · final quiz	workbook

ONE WARNING BEFORE YOU START

Almost every number in this competency is a negotiation, not a calculation. An availability target, a canary population, a metric threshold — each is a claim about what users will tolerate, agreed by people who can be wrong. If you finish able to compute an error budget but unable to say *who has to agree to it and what they give up*, the booklet has failed.

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Given a real reliability decision (“what should our target be?”, “we blew the budget, now what?”, “is this release safe to push?”, “users are unhappy but we’re inside SLO”), you can:

- 1. **Name the instrument** — indicator, objective, budget, policy — and say which one the argument is actually about.
- 2. **Locate the disagreement** between what you measure and what users feel, and say which of the two is wrong.
- 3. **Prescribe the move** — tighten, loosen, change the implementation, make it aspirational, canary it — and name the near-miss you rejected.
- 4. **State the price.** Every one of these costs engineering time, feature velocity, customer experience, or credibility.

The core model in one paragraph

Perfect reliability is unbuyable and, past a point, unwanted: each extra nine costs more while its value to the customer approaches zero, and a system that can never change dies slowly. So you pick a target below 100%, and the gap between it and perfection becomes a **budget** — a quantity of unreliability you may spend. That converts an argument about opinions into arithmetic: what did this outage cost, is this project worth more than that one, may we ship today. And once unreliability is a currency, the question stops being how to avoid spending it and becomes **how to spend it well** — which is what canarying is: information about a release, bought at a small bounded price.

THE WHOLE THING FOR A SMART 15-YEAR-OLD

- Nothing is perfect, and chasing perfect costs more and more for less and less.
- So decide how much failure is acceptable, and treat the rest as an allowance you're allowed to spend.
- Now arguments about "is it reliable enough?" become arithmetic, and everybody has to agree the number in advance.
- Then spend a little of that allowance on purpose — show a new version to a few users first — so you find out it's broken cheaply instead of expensively.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 Why 100% is wrong	The four reasons perfection is wrong; the budget that falls out of a sub-100% objective; who owns it; the preconditions without which none of this works.	Everything else depends on accepting a number below perfection. Until that's settled there is no budget and nothing to spend.
2 What to measure	Indicators as good-over-total ratios; specification versus implementation; the three component types; the worked example; rounding measurements into a first objective; time windows.	A budget is only as honest as the measurement behind it. Get the indicator wrong and every later decision is confidently wrong.

3 Making it bite	Stakeholder agreement, the error budget policy, improving coverage, recall and precision, aspirational objectives, and the decision matrix.	An objective nobody acts on is a reporting metric. This part turns a number into a decision-making tool.
4 Spending it to learn	Release engineering principles, canary and control, population and duration, metric selection, isolation, monitoring requirements, and the three things that resemble canarying but aren't.	With a budget that bites, the last question is how to spend it deliberately rather than lose it by accident. A canary is the cheapest information you can buy.

“One could even claim that without SLOs, there is no need for SREs.”

1

PART 1 OF 4

Why 100% is the wrong target

The objective below perfection, the budget that falls out of it, and who has to own the trade.

TENTH-GRADER VERSION

- Aiming for "never broken" sounds responsible. It's actually the most expensive possible goal, and you still won't hit it.
- Your users don't experience your system directly — there's a long chain of other things between you and them, any of which can break.
- If you promise perfection you can never change anything, and the biggest cause of outages is change.
- So pick a number below perfect. The gap between that number and perfect is an allowance you're allowed to spend.

Start with the delivery promise

- A courier can promise “next day, every time, no exceptions.” To keep it, they need spare vans, spare drivers, spare depots — and even then a bridge closure defeats them.
- Going from 99% to 99.9% on-time means buying a second depot. From 99.9% to 99.99% means a third. **Each nine costs more than the last**, while the customer's happiness barely moves — nobody notices the difference between one late parcel a year and one every ten years.
- And a courier who promises perfection can never reroute, never try a faster van, never change anything — because every change risks the promise.
- So they promise something achievable and keep the difference as room to manoeuvre. That room is the whole idea.

The four reasons perfection is wrong

REASON	WHAT IT ACTUALLY SAYS
You can't reach it	Even with redundant components, automated health checking and fast failover, there is a nonzero probability that one or more components fail simultaneously — which puts availability under 100%.
Users wouldn't see it anyway	Even if your system were perfect, the chain of systems between you and your customers is long and complex , and any of it can fail. So as you go 99% → 99.9% → 99.99%, each extra nine costs more while the marginal utility to your customers steadily approaches zero .
You could never change	To hold perfection you can never update or improve. And the number one source of outages is change — new features, security patches, new hardware, scaling to meet demand. Sooner or later the service stagnates and customers leave, which is not great for anyone's bottom line.
It makes you purely reactive	At 100% you can do nothing but react to the sub-100% availability that is guaranteed to happen. Stated sharply: reliability of 100% is not an engineering culture objective — it's an operations team objective .

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
------	------------------	------------------	---------------------------------------

Service level objective SLO	A target level of reliability for the service's customers. Above it, almost all users should be happy; below it, they start complaining or leave.	97% availability. 90% of requests faster than 450 ms.	vs a service level agreement (SLA): an SLA is a <i>business contract</i> that comes into effect when users are so unhappy you have to compensate them. An SLO is an internal target you set before anyone is owed money.
Service level indicator SLI	An indicator of the level of service you are providing — the thing you measure.	The proportion of HTTP requests that succeeded.	vs the objective : the indicator is the measurement, the objective is the target you hold it to. One indicator can carry several objectives at different thresholds.
Error budget	100% minus the SLO — the quantity of unreliability you are permitted to spend in a period.	A 99.9% success-ratio objective on a service taking 3 million requests over four weeks gives a budget of 3,000 errors . A single outage causing 1,500 of them costs 50% of the budget .	vs uptime or a reporting metric: a budget is spendable. If nobody may act on its exhaustion it isn't a budget, it's a number on a dashboard.
Error budget policy	A written statement of what happens, and who does it, when the budget is exhausted.	Development gives top priority to reliability bugs from the past four weeks; or focuses exclusively on reliability until back in objective; or a production freeze halts certain changes.	vs the objective itself: the objective says what “good” is; the policy is what gives it teeth. Getting the policy approved is also the best test of whether the objective is fit for purpose.
Reliability	How the service is performing with regard to <i>all</i> of its indicators.	Could mean availability, latency, or several at once.	vs availability alone, which is one indicator among many. Freshness, durability, correctness, quality and coverage all count as reliability too.

WHO OWNS THE NUMBER

Once the objective is below 100%, it must be **owned by someone empowered to make tradeoffs between feature velocity and reliability**. In a small organisation that may be the CTO; in a larger one it is normally the product owner or product manager. An objective owned by the people who merely operate the service is an objective nobody can trade against, which means it will lose every argument it has with a launch date.

The four preconditions — without these, none of it works

To adopt an error-budget-based approach, all four must hold:

- 1. There are objectives that **all stakeholders have approved** as fit for the product.
- 2. The people responsible for meeting them **agree it is possible** to do so under normal circumstances.
- 3. The organisation has **committed to using the budget** for decision making and prioritising, formalised in an error budget policy.
- 4. There is a **process for refining** the objective.

Otherwise, “SLO compliance will simply be another KPI (key performance indicator) or reporting metric, rather than a decision-making tool.”

Three places teams actually start from

MATURITY LEVEL	WHAT IT LOOKS LIKE
Greenfield	Nothing deployed yet. Nothing to measure, and no habits to unlearn.
Production, no objectives	Monitoring exists and notifies you when things go awry, but there are no formal objectives, no concept of an error budget, and an unspoken goal of 100% uptime .
An objective without teeth	A running deployment with an objective below 100%, but no common understanding of its importance or how to use it for continuous improvement. The number exists; nothing follows from it.

WHY THIS IS A PRIORITISATION TOOL, NOT A REPORTING ONE

Engineers are scarce even at the largest organisations, so engineering time should go to the most important characteristics of the most important services. The budget makes that comparison arithmetic rather than rhetoric: given two candidate reliability projects — say **automating rollbacks** versus **moving to a replicated data store** — you estimate each one's impact on the error budget and fund the one that buys more. Day to day, the work of the people defending reliability is driven by the objectives: defending them in the short term, and keeping them defensible in the medium to long term.

CONCEPT BOUNDARY — WHAT AN ERROR BUDGET IS NOT

It is not a licence to be careless, and it is not a measure of how much downtime you have “left” before someone is angry.

It is a decision-making instrument: a quantity you can attribute to specific incidents and specific projects, so that arguments about what to fund become comparisons between numbers rather than between opinions. Its authority comes entirely from the fact that everyone agreed to it in advance.

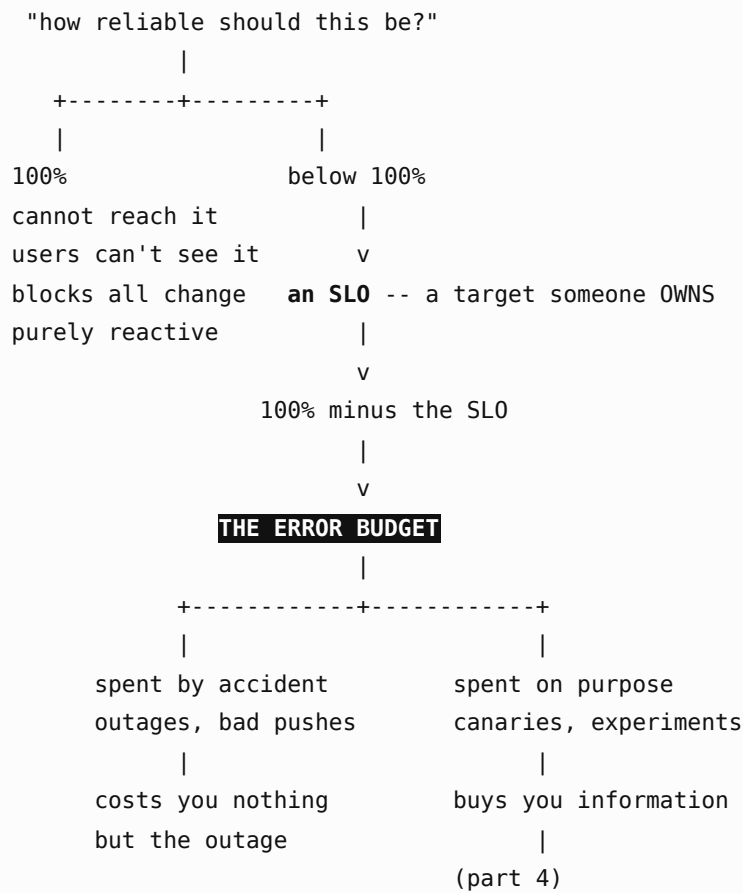
Anchor sketch — where the budget comes from

Legend:

boxed = a named node

reversed = the node that matters most

Q = cover the page and answer this



Q Name the four reasons the left branch is wrong, and sort them: which is about *your* system, which is about the chain outside it, and which two are about your organisation's behaviour?

Q What has to be true about the reversed node before it is a budget rather than a dashboard number?

Q Who must own the objective, and what specifically are they empowered to trade?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“We aim for no downtime at all.”	“That's an unspoken 100% target, which is unreachable, invisible to users behind a long dependency chain, and blocks every change — and change is the number one source of outages.”
“How much downtime are we allowed?”	“The error budget is 100% minus the objective, expressed in the same units as the indicator — so for a 99.9% success ratio over 3 million requests, 3,000 errors.”
“That outage was pretty bad.”	“That incident consumed 50% of the four-week error budget, which puts it ahead of everything else on the reliability backlog.”
“We have a target but nobody really uses it.”	“An objective without teeth. Without a committed error budget policy it's a KPI, not a decision-making tool.”
“Ops owns the reliability number.”	“It has to be owned by someone empowered to trade feature velocity against reliability — normally the product owner, or the CTO in a small organisation.”
“Should we do the rollback automation or the replicated store?”	“Estimate each one's impact on the error budget and fund the one that buys more of it back.”
“We promise 99.9% in the contract.”	“That's an SLA — a business contract with compensation attached. The internal objective should sit tighter than the agreement, so you notice before you owe anyone money.”

Retrieval — cover the page first

PART 1 · QUESTION 1

A payments team has run at “100% uptime, no exceptions” for two years. They are proud of it, and they are genuinely good: last year they had eleven minutes of unplanned downtime. They also ship roughly once a quarter, because every release is a risk to the promise, and the on-call rota is exhausted.

(a) Name the three distinct costs this target is imposing, mapping each to one of the reasons perfection is the wrong goal. **(b)** Explain why their excellent track record makes the problem harder to argue, not easier. **(c)** Name the first thing you'd establish — not the number itself — and say who has to own it and what they must be empowered to do.

[illegible]

PART 1 · QUESTION 2

An analytics service publishes a 99.5% availability objective on a wiki. It is measured, charted monthly, and reported to the leadership team. Last quarter it came in at 98.8%. Nothing happened: no work was reprioritised, no release was held, and the next quarter's roadmap was unchanged.

(a) Name what they have, precisely, and say what it is missing. **(b)** Of the four preconditions for an error-budget-based approach, identify which ones are unmet and why the failure of any one is sufficient. **(c)** Give the artefact you would create next, list three things it must specify, and name the near-miss fix — publishing the number more prominently — and why it changes nothing.

2×2 — is your objective actually doing anything?

COLUMNS: HAS THE ORGANISATION COMMITTED TO ACT WHEN THE BUDGET RUNS OUT? →

	No commitment nothing follows from the number	Committed policy named actions, named owners
TARGET IS 100% stated or unspoken	The unspoken-perfection shop Monitoring notifies you when things go awry; no formal objective, no budget, and an unstated goal of never failing. <i>Move:</i> the first task isn't measurement, it's agreeing that a number below 100% is acceptable. Until then there is nothing to spend.	Structurally impossible You cannot have a policy for exhausting a budget of zero. At 100% you can only ever react to the sub-100% that is guaranteed to happen. <i>Move:</i> recognise this cell as a contradiction, not a goal. It's an operations posture wearing an engineering label.
TARGET IS BELOW 100%	An objective without teeth The number exists and is charted. Nobody has agreed what happens when it's missed, so nothing does. <i>Move:</i> write and get approval for the error budget policy. Approval is also the test — if stakeholders won't sign, the objective is wrong, not the policy.	A budget you can spend All stakeholders approved it, the defenders agree it's achievable under normal conditions, the policy is committed, and there's a process for refining it. <i>Move:</i> now attribute incidents to budget consumed, fund projects by budget bought back, and spend the remainder deliberately — which is part 4.

Read it as: the left column is where most organisations live, and both cells fail for the same reason — the number cannot cause anything to happen. Moving right is an organisational act, not a technical one; moving down is a cultural one.

CARRY-OUT RULE FROM THIS PART

A target nobody may act on is a report, not an objective. Before arguing about whether the number should be 99.9% or 99.95%, settle who is allowed to stop a release when it's missed. If the answer is nobody, the number is decoration whatever its value.

What to measure, and how to get a first number

Good events over total events, specification versus implementation, and rounding reality into an objective.

TENTH-GRADER VERSION

- Measure reliability as a fraction: things that went well, divided by all the things.
- That always gives you a number between 0 and 100, which makes everything downstream easier.
- There's a difference between *what* you want to measure and *how* you actually measure it — and the how is where all the compromises hide.
- Your first number doesn't have to be right. It has to exist, and it has to be improvable.

Start with the bus route

- You want to know if the bus route is any good. “Buses that arrived within five minutes of schedule, divided by all scheduled buses” — a fraction, 0 to 100%.
- Now: **how do you count it?** The driver's log is free but misses buses that never left the depot. An inspector at one stop is more accurate but covers one stop. A GPS unit on every bus is best and costs the most.
- Same question — “are buses on time?” — three different instruments, each with different **quality** (how well it reflects a passenger's experience), **coverage** (how many passengers it speaks for), and **cost**.
- That gap between the question and the instrument is the single most important idea in this part.

The shape of an indicator

- Treat the indicator as **the number of good events divided by the total number of events**. Examples:
 - Successful HTTP requests / total HTTP requests (a success rate).
 - gRPC calls that completed successfully in under 100 ms / total gRPC requests.
 - Search results that used the entire corpus / total search results, including those that degraded gracefully.
 - “Stock check count” requests from product searches that used stock data fresher than 10 minutes / total stock check requests.
 - “Good user minutes” by some extended list of criteria / total user minutes.
- Two properties make this shape worth insisting on:
 - It **ranges 0% to 100%**, where 0% means nothing works and 100% means nothing is broken. Intuitive, and it lends itself directly to a budget — the objective is a target percentage and the budget is 100% minus it.
 - Consistency **buys you tooling**. Alerting logic, analysis tools, budget calculation and reports can all expect the same three inputs: numerator, denominator, threshold.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
------	------------------	------------------	---------------------------------------

SLI specification	The assessment of service outcome you think matters to users, independent of how it is measured.	“Ratio of home page requests that loaded in under 100 ms.”	vs implementation : the specification is the question; the implementation is the instrument. One specification can have several implementations with genuinely different answers — which is why arguments about “our availability number” are usually arguments about implementation.
SLI implementation	The specification plus a concrete way to measure it.	The same ratio, measured from the latency column of the server log; or by probers running JavaScript in a browser in a VM; or by instrumentation in the page's own JavaScript reporting to a telemetry service.	Each carries its own blind spot. Logs miss requests that never reach the backend . Probers catch unreachable-network errors but may miss issues affecting only a subset of users . Client instrumentation captures experience most accurately but needs code changes and new infrastructure — which then has its own reliability requirements .
Quality, coverage, cost	The three axes an implementation trades on: how accurately it captures one customer's experience; how well it captures <i>all</i> customers' experience; and what it costs to build.	Client-side instrumentation is high quality; a periodic prober is low coverage; server logs are cheap.	Quality and coverage are different : an instrument can perfectly capture the experience of the 2% of users it watches. Confusing them is how teams end up confident in a number that speaks for almost nobody.
Request-driven	A component where the user creates an event and expects a response.	An HTTP service behind a browser; an API for a mobile app.	Its indicators are availability, latency, quality — where quality means the proportion of responses served <i>undegraded</i> , which only matters if the service degrades gracefully rather than failing.
Pipeline	A system that takes records in, mutates them, and puts the output somewhere else.	Reading a relational database into a distributed hash table for serving; converting video formats; reading logs from many sources to generate reports; a monitoring system pulling metrics to generate time series and alerts.	Its indicators are freshness, correctness, coverage . Note the range: this might be a simple real-time process on one instance, or a multistage batch job taking many hours — both are pipelines.
Storage	A system that accepts data and makes it retrievable later.	Bytes, records, files, videos.	Its indicator is durability — and it carries a trap. If you hold a billion records over ten years but the user wants only today's, and today's are unavailable, they are unhappy even though almost all their data is readable . A durability number averaged over everything you store can be excellent and meaningless.

The worked example — a mobile phone game

- The app on the phone talks to an **HTTP API** in the cloud, which writes state changes to **permanent storage**. A **pipeline** periodically runs over that data to generate league tables — high scores for today, this week, all time — writing to a separate league table store, served to both the mobile app and a website. Users upload custom **avatars**, used in-game and on the high-score site.
- All three component types are present, so all seven indicator types are candidates. The guidance is to choose **five or fewer** indicator types representing the most critical functionality.
- **Availability and latency, API and web:** success is based on HTTP status code — 5XX counts against, everything else counts as successful. Candidate sources: application server logs, load balancer monitoring, black-box monitoring, client-side instrumentation. The example picks **load balancer monitoring**, for two reasons worth remembering together: the metrics were *already available*, and they are *closer to the user* than the application server's logs.
- **Pipeline freshness:** the pipeline records a **watermark** — the timestamp of when the data was updated. Two implementations: query the league table periodically counting fresh records against total, which treats every stale record as equally important regardless of how many users saw it; or have clients check the watermark on request and increment counters. The example picks the **client-side** version, because it correlates far more closely with user experience and is straightforward to add.
- **Pipeline coverage:** the pipeline exports how many records it *should* have processed and how many it *did*. Known blind spot: it **may miss records the pipeline never knew about** because of misconfiguration.
- **Pipeline correctness:** either inject data with known outputs and count how often output matches expectation, or compute correct output by a method distinct from the pipeline (likely more expensive, hence unsuitable for the pipeline itself) and sample input/output pairs. The example uses **manually curated data with known good outputs**, tested on every pipeline run — with the standing caveat that the curated data must be **representative of real-world data** or the indicator is representative of nothing.

MULTIPLE GRADES FOR ONE INDICATOR

A single threshold hides the tail. If 90% of requests return within 100 ms but the remaining 10% take ten seconds, many users are unhappy while the number looks fine. So set **several thresholds on the same indicator**: 90% of requests faster than 100 ms, *and* 99% faster than 400 ms. This applies to any indicator with a parameter that measures user unhappiness.

From measurement to a first objective

- Measure what the service actually does, then **round down to manageable numbers** — roughly two significant figures of availability, and up to 50 ms of latency. (50 ms because users are unlikely to perceive a 50 ms change; the right window depends on the service — a reporting service is not a real-time game.)
- Over four weeks the example API showed: **3,663,253 total requests; 3,557,865 successful (97.123%); 90th percentile latency 432 ms; 99th percentile 891 ms**.
- Rounded, that gives the proposed objectives and their budgets:

OBJECTIVE	ALLOWED FAILURES OVER FOUR WEEKS
97% availability	109,897
90% of requests faster than 450 ms	366,325
99% of requests faster than 900 ms	36,632

- Note the deliberate consequence: **if you set the objective from actual measurements, by design you met it.** That is fine — the value is in what the distribution then tells you. Were there days in the past four weeks when the service missed it? Do those days correlate with real incidents? Was action taken, or should it have been?
- On measuring latency: **it is better to count the slow requests than to approximate them with a histogram** — but if only histogram buckets are available, use them. Counting explicitly against configured thresholds is more accurate but needs more configuration, and makes changing the thresholds retroactively harder.
- If you have no logs, metrics or historical performance at all, configure a source: for HTTP services a remote monitoring service doing a periodic ping or GET and reporting successes is a legitimate low-fidelity start.

CONCEPT BOUNDARY — CURRENT PERFORMANCE AS A STARTING POINT

The standing advice is not to set an objective from current performance, because it can commit you to unnecessarily strict targets.

But current performance is a good place to start if you have nothing else and a real process for iterating. The trap runs the other way too: **don't let current performance limit you.** Customers come to expect the service to perform as it does, so if you currently return successes 99.999% of the time in under 10 ms, any significant regression from that baseline may make them unhappy — whatever the published objective says.

Choosing the time window

CHOICE	WHAT IT BUYS	WHAT IT COSTS
Rolling window	Closer to user experience: if you have a large outage on the last day of a month, your user doesn't suddenly forget about it on the first day of the next.	Define it as an integral number of weeks so it always contains the same number of weekends — a 30-day window may hold four or five, and if weekend traffic differs your indicator moves for uninteresting reasons.
Calendar window	Closer to business planning and project work — evaluate quarterly to decide where the next quarter's headcount goes.	Introduces uncertainty: mid-quarter you cannot know how many requests remain, so decisions must speculate about how much budget you'll spend in the rest of the period.
Shorter windows	Faster decisions. Miss last week and small course corrections — prioritising the relevant bugs — can avoid violations in future weeks.	Too little data to justify anything large.
Longer windows	Strategic decisions: choosing between a high-availability distributed database, automating rollout and rollback, or deploying a duplicate stack in another zone.	Slow feedback. The rule of thumb: the amount of data required is roughly commensurate with the amount of engineering work being proposed to fix it.

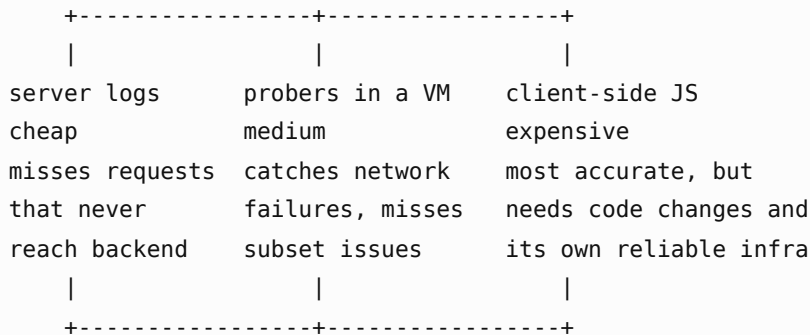
The general-purpose recommendation: a **four-week rolling window**, complemented by weekly summaries for task prioritisation and quarterly summarised reports for project planning.

Anchor sketch — one question, several instruments

Legend: boxed = a named node reversed = focus node Q = self-test cue

SLI SPECIFICATION "what outcome matters to users?"
e.g. ratio of home page loads under 100 ms

one question, several instruments



each differs in

QUALITY . COVERAGE . COST

pick the cheapest one that is close enough to the user -- then iterate

Q Quality and coverage are different things. Give an instrument that is high on one and low on the other.

Q The worked example chose load balancer monitoring. Give *both* reasons, not just the obvious one.

Q What extra obligation does the most accurate instrument create that the other two don't?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
"How often does it work?"	"Express it as good events over total events, so it lands between 0 and 100% and the budget is just 100% minus the objective."
"We measure it from the server logs."	"That's one implementation of the specification. It's cheap, but it misses requests that never reached the backend, so it overstates availability."
"Our monitoring covers everything."	"Distinguish quality from coverage — an instrument can capture 2% of users perfectly. Which is it doing?"
"The average is fine."	"Averages hide the tail. Set multiple grades on the same indicator — 90% under 100 ms and 99% under 400 ms."
"Is the data up to date?"	"That's a freshness indicator. Have the pipeline write a watermark and let clients check it on read, so the measure is weighted by who actually saw stale data."
"We hit our number, so we're fine."	"We set it from measured performance, so by design we met it. The interesting question is which days missed it and whether they correlate with real incidents."
"Let's measure it monthly."	"Prefer a four-week rolling window — an integral number of weeks, so weekend traffic doesn't move the number for uninteresting reasons."

PART 2 · QUESTION 1

A team reports 99.95% availability from application server logs. Support is receiving steady complaints that the site “doesn't load”, concentrated among mobile users on one carrier. Engineering cannot reproduce it and points at the dashboard.

(a) Explain the discrepancy in terms of specification versus implementation, and name the specific blind spot. **(b)** Give two alternative implementations of the same specification and state what each would and would not catch. **(c)** Say which you'd adopt first and why, and name the extra obligation the most accurate option creates.

.....

.....

.....

.....

.....

.....

PART 2 · QUESTION 2

A nightly job rebuilds a product recommendations table. The team proposes a single indicator: “the proportion of nightly runs that completed without error”, with an objective of 99%. Users complain that recommendations are sometimes days out of date, and occasionally plain wrong — but the job has completed successfully every night for a month.

(a) Name the component type and the three indicator types appropriate to it, and say which the team has actually implemented. **(b)** For each of the two user complaints, name the missing indicator and describe a concrete implementation. **(c)** Name the blind spot the coverage implementation retains even when built correctly, the standing caveat on the correctness one, and say why simply tightening the existing “runs completed without error” objective would fix neither complaint.

.....

.....

.....

.....

.....

.....

2×2 — choosing an instrument

	Cheap already available, or nearly	Expensive new code, new infrastructure
FAR FROM THE USER	Server-side logs Success and latency as the application saw it. <i>Move:</i> a legitimate first instrument — if logs exist and probers would take weeks, use the logs. Just state the blind spot out loud: requests that never reached the backend are invisible, so the number flatters you.	Elaborate back-end telemetry Expensive to build, still measuring your side of the wire. <i>Move:</i> the worst trade on the grid. If you are spending real engineering effort, spend it moving <i>closer to the user</i> , not on more detail about a vantage point you already have.
CLOSE TO THE USER	Load balancer monitoring The example's choice, for two reasons at once: the metrics were already available, <i>and</i> they sit closer to the user than application logs. <i>Move:</i> look here first. The best first instrument is usually one you already own that happens to sit further out than where you were looking.	Client-side instrumentation Probers running real JavaScript, or telemetry from the page itself. <i>Move:</i> highest quality, and the honest destination — but it needs code changes, new infrastructure, and that infrastructure has its own reliability requirements . Earn your way here; don't start here.

Read it as: the vertical axis is quality, the horizontal is cost — and **coverage is a third axis this grid doesn't show**. A prober can be close to the user and cheap while speaking for almost nobody, so always ask the coverage question separately before trusting a cell.

CARRY-OUT RULE FROM THIS PART

Get something measured, then improve it. Your first indicator does not have to be correct — it has to exist and to be attached to a feedback loop. Pick the aspect that is relevant and easy to measure, name its blind spot in writing, and iterate from there.

TENTH-GRADER VERSION

- A number only matters if people agreed to it beforehand and agreed what happens when it's missed.
- Getting that agreement is also the best test of whether the number is any good — if nobody will sign, it's wrong.
- Check your number against reality: when users complained, did your measurement notice?
- And when your measurement and your users disagree, one of them needs changing — usually the measurement.

Start with the household budget

- A household agrees to spend no more than a set amount each month. The number is easy. What makes it real is two other things: **everyone agreed it in advance**, and **everyone agreed what happens if it's blown** — no takeaways until next month.
- The agreement is also the test. If one person refuses to sign because the number is impossible on their salary, the *number* is wrong, not the person.
- And once a year you check it against reality: did the months you felt broke line up with the months you overspent? If they didn't, the budget is measuring the wrong thing.

Who has to agree, and what each is agreeing to

PARTY	WHAT THEY MUST AGREE	WHAT THEIR REFUSAL TELLS YOU
Product managers	That this threshold is good enough for users — that performance below it is unacceptably low and worth spending engineering time to fix.	If they think users will suffer badly before the policy prompts anyone to act, the objective isn't tight enough .
Product developers	That if the budget is exhausted, they will take steps to reduce risk to users until the service is back in budget.	If the reliability work would push feature velocity below acceptable levels, they can argue to relax the objective — and the product manager must understand the trade, because lowering the objective also lowers the number of situations the reliability team responds to .
The team defending it	That the objective is defensible without Herculean effort, excessive toil, and burnout — all of which damage the long-term health of the team and the service.	If it isn't defensible, they can make a case for relaxing some objectives. This is a legitimate argument, not a failure of nerve.

If all three parties will not agree to enforce the policy, iterate on the indicators and objectives until they will. Decide what you need to move forward: **more data, more resources, or a change to the indicator or the objective**.

THE POLICY IS THE TEST

Getting the error budget policy approved by the product manager, the development team and the reliability team is a **good test for whether the objectives are fit for purpose**. Three different refusals point at three different defects, and each is informative. And if a service does exhaust its budget but not all stakeholders agree that enacting the policy is appropriate, that isn't an argument to have in the moment — **you return to the policy approval stage**.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Recall	The proportion of significantly user-impacting events that the indicator captures.	Users had a bad hour; did the indicator dip?	vs precision : recall is about missing real problems; precision is about raising false ones. They trade against each other, which is the whole difficulty.
Precision	The proportion of events captured by the indicator that were significantly user-impacting.	The indicator dipped; did anyone actually suffer?	The governing warning: it is pointless to improve recall if you lower precision so far that the team must constantly respond to unimportant events . An alert nobody believes is worse than no alert.
Aspirational objective	A tighter target measured and tracked alongside the current one, but explicitly called out in the policy as not requiring action .	You determine users need a tighter target, but reaching it will take months of product work.	vs simply tightening the objective : tightening now would leave you permanently out of budget and subject to the policy — a perpetual state of emergency. The aspirational form lets you track progress without crying wolf.
Critical user journey	A sequence of tasks that is a core part of a user's experience and an essential aspect of the service.	For online shopping: searching for a product; adding a product to a cart; completing a purchase.	These almost certainly won't map well to existing indicators — each is multiple complex steps that can fail anywhere, and inferring success from logs is extremely difficult. (Did the user fail at step three, or get distracted by cat videos in another tab?) Their value: they improve recall without affecting precision .
Bucketing	Adding labels to indicators and applying different objectives to different labels.	By tier: premium 99.99%, free 99.9%. By expected responsiveness: interactive requests that block page load, 90% within 100 ms; CSV downloads, 90% start within 5 s.	vs one objective for everything: not all requests are equal — a notification check from a mobile app matters, but not as much as an advertiser's billing request. Caveat on per-customer tracking: low-volume customers land at either 100% or very low availability , so individual misses are often uninteresting while aggregate patterns are a useful signal.
Critical dependency	A dependency whose unavailability means <i>your</i> service is also unavailable.	A component supporting a high-value interaction.	Its reliability guarantee must be at least as high as that of the dependent action, and the team running it owns its objective the same way you own the product's. If the component can't reach the level you need, you must engineer around it — a different component, or defences like caching, offline store-and-forward, graceful degradation.

Checking the number against reality

- First you need a source of information about user satisfaction. The options are wide: **count outages discovered manually, posts on public forums, support tickets, calls to customer service**; attempt to measure sentiment on social media; add code to sample user happiness periodically; run face-to-face surveys. Start with something **cheap to collect** and iterate. Asking the product manager to fold reliability into their existing customer conversations about pricing and functionality is an excellent place to start.
- Then correlate. Look at periods with a known outage — do they line up with steep drops in error budget? Look at times the indicator dipped — do they line up with known outages or a rise in support tickets? (If you're comfortable with statistics, **Spearman's rank correlation coefficient** quantifies this.)
- The outliers are the interesting part. In the worked graph of tickets per day against budget lost, two points stand out: **one day with only 5 tickets where 10% of the budget was lost**, and **one day with 40 tickets where no budget was lost at all**. Both warrant closer investigation — the first is a precision problem, the second a recall problem.
- If outages and ticket spikes aren't captured by any indicator, or you have dips and misses that don't map to user-facing issues, **your objective lacks coverage**. This is normal and expected. Indicators and objectives should change as the service changes.

Four moves when the objective lacks coverage

MOVE	WHEN	HOW
Change the objective	The indicator showed a problem but the objective didn't prompt anyone to notice or respond.	Take the indicator values during the periods of interest, calculate what objective <i>would</i> have triggered a notification then, apply it to historic indicators, and see what else it would have caught. For false-positive days, consider relaxing instead.
Change the implementation	Changing the objective in either direction produces too many false positives or false negatives.	Two directions only: move the measurement closer to the user (server → load balancer → client), or improve coverage so you capture more interactions (a simple GET → a health handler exercising more functionality → a test executing all the client-side JavaScript).
Institute an aspirational objective	You need a tighter target, but improving the product to meet it will take time.	Measure and track it alongside the current one, and say explicitly in the policy that it does not require action.
Iterate	Always.	Pick the option most likely to give the highest return. Especially in the first few rounds, err on the side of quicker and cheaper — that reduces uncertainty in your metrics and tells you whether you need the expensive ones at all.

Deciding with the budget

- **Size incidents by budget consumed.** A release causing 100% `NullPointerExceptions` for four hours before revert produced 14,066 errors — against a budget of 109,897, that single event used **13%**. A singly-homed state database failing, with 20 hours to restore from backups, caused an estimated 72,000 errors — **65%** of the budget.
- **Then compare classes of incident, not instances.** One server failure in five years, versus two or three bad releases needing rollback per year. On average, **bad pushes cost twice as much error budget as database failures**. The numbers say to address the release problem rather than investigate the server. That comparison is the entire point of the instrument.
- **In extremis**, a team can declare an emergency with high-level approval to deprioritise all external demands until exit criteria are met — typically that the service is within objective *and* that steps have been taken to reduce the chance of a repeat: improving monitoring, improving testing, removing dangerous dependencies, or rearchitecting to remove known failure types.

- **And in the happy case:** if the service runs flawlessly and needs little oversight, move it to a less hands-on tier of support. Keep incident response and high-level oversight; redirect the attention to systems that need it.

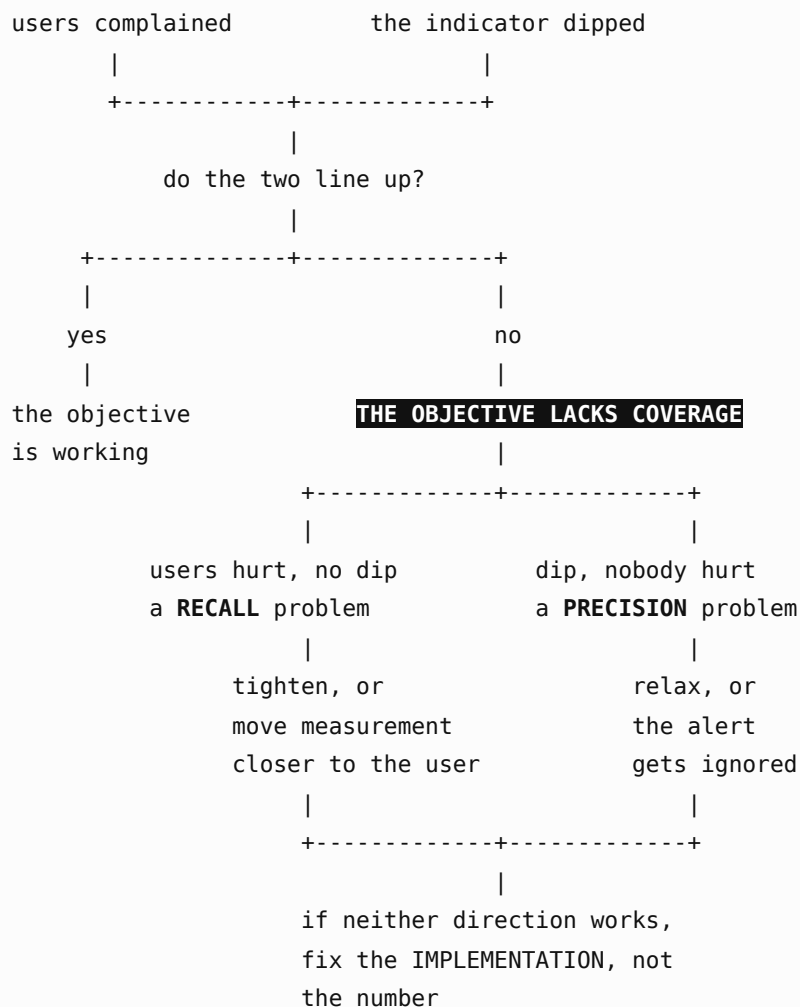
CONCEPT BOUNDARY — TWO THINGS THE BUDGET IS HONEST ABOUT NOT KNOWING

Shape. The budget is an approximation of user satisfaction. A four-hour outage every 30 days probably produces fewer unhappy users than four separate one-hour outages, which in turn produces fewer than a constant 0.5% error rate — **but the budget treats all three the same.** Where that matters, you need thresholds tuned to the service.

Independence. It is tempting to do arithmetic: a service at 99.9% in one zone, deployed in two zones, “should” give 99.9999%. That assumes the two are wholly independent, which is **almost never the case** — common dependencies, common failure domains, shared fate, global control planes. Unless every one is enumerated and accounted for, **any such calculation is deceptive.**

Anchor sketch — when measurement and users disagree

Legend: boxed = a named node reversed = focus node Q = self-test cue



Q Define recall and precision in one line each, and say why improving one can make the system worse overall.

Q Name the two — and only two — directions in which an implementation can be improved.

Q You need a tighter target but can't meet it for six months. What do you do instead of tightening, and what does the policy have to say about it?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“What do we do when we miss it?”	“That's the error budget policy — the specific actions taken on exhaustion and who takes them, approved in advance by product, development and the reliability team.”
“They won't sign off on the target.”	“Then the target is wrong, not the signatory. Which party is refusing tells you whether it's too loose, too tight, or undefendable without excessive toil.”
“Our alert didn't fire but users were angry.”	“A recall problem — the indicator failed to capture a user-impacting event. Tighten the objective or move the measurement closer to the user.”
“It keeps alerting on nothing.”	“A precision problem. And it's worse than it looks — chasing recall at the cost of precision produces alerts the team learns to ignore.”
“We want a tighter number but can't hit it yet.”	“Make it an aspirational objective — tracked alongside, explicitly excluded from the error budget policy, so you're not in permanent emergency.”
“Which reliability project should we fund?”	“Compare classes of incident by budget consumed. If bad pushes cost twice what database failures do, fund the release problem.”
“We'll just run it in two zones.”	“That multiplication assumes independence, which almost never holds — shared dependencies, shared failure domains, shared fate, global control planes.”
“The outage was someone else's fault.”	“There are two schools on dependency-caused misses. Freezing anyway makes users happier — and whichever you pick, record it in the documented error budget policy.”

PART 3 · QUESTION 1

A checkout service has met its 99.9% availability objective every month for a year. Support ticket volume has doubled over the same period, and the product manager reports that customers describe checkout as “flaky”. Investigation shows most complaints involve the payment step failing after the cart page loaded fine; each individual HTTP request in that flow succeeded.

(a) Name the problem in the precision/recall vocabulary and say which one it is. **(b)** Explain why an objective composed of per-request availability can be met while a multi-step flow fails, and name the concept that addresses it. **(c)** Give the property of that concept that makes it especially attractive here, name the practical difficulty of implementing it, and say why the near-miss fix — tightening the existing per-request objective — would not catch these failures.

PART 3 · QUESTION 2

A team blows its error budget in week two of a four-week window because an upstream identity service — owned by another team, in another division — had a six-hour outage. Their own code performed perfectly throughout. The development lead argues that a feature freeze would be punishing them for someone else's failure.

(a) State both positions on this fairly, and say which one the guidance favours and on what grounds. **(b)** Say where this decision should have been made and recorded, and why deciding it now is the wrong time. **(c)** Name the structural fix for the identity service's reliability, and two defensive measures if that fix isn't available.

2x2 — the decision matrix, collapsed

COLUMNS: ARE CUSTOMERS ACTUALLY SATISFIED? →

	Satisfaction HIGH	Satisfaction LOW
OBJECTIVE MET	It's working <i>Low toil:</i> relax release and deployment processes and increase velocity, or step back and move engineering attention to services that need it. <i>High toil:</i> if alerting is generating false positives, reduce sensitivity; otherwise temporarily loosen the objective (or offload the toil) and fix the product or improve automated fault mitigation.	Your objective is lying to you You are passing a test your users are failing. Toil level doesn't change the answer here — high or low, the move is the same. <i>Move:</i> tighten the objective. Then check whether tightening alone is enough, or whether the implementation is measuring the wrong thing entirely.
OBJECTIVE MISSED	Too strict You're failing a test your users don't care about. Again toil doesn't change the answer. <i>Move:</i> loosen the objective. Holding a number users don't feel burns credibility and engineering time at the same rate.	Genuinely in trouble <i>Low toil:</i> increase alerting sensitivity — you're missing things you could be catching. <i>High toil:</i> offload the toil and fix the product, or improve automated fault mitigation. This is the cell where more engineering is actually the answer.

Read it as: the diagonal cells, where the objective and the users **agree**, are the ones where toil decides what to do. The off-diagonal cells, where they **disagree**, have one answer regardless of toil — and the answer is always to change the number, because the number is the thing that's wrong.

CARRY-OUT RULE FROM THIS PART

When your measurement and your users disagree, the measurement is the defect. Users are the ground truth by definition — the objective exists to predict their happiness. So tighten, loosen, or move the instrument closer to them, and treat “but we hit our number” as a symptom rather than a defence.

Spending the budget to buy information

Canary and control, population and duration, metric selection — and the three things that look like canarying but aren't.

TENTH-GRADER VERSION

- Show the new version to a small slice of real users first, and compare them against everyone else.
- If it's broken, only that slice suffers — and you find out for a fraction of the cost.
- The comparison has to be against people using the system *at the same time*, not against yesterday.
- And you have to pick the right things to watch, or you'll either miss real breakage or cry wolf until everyone ignores you.

Start with the coal mine

- Miners carried a canary underground. The bird is smaller and breathes faster than a human, so it is **intoxicated by dangerous gases faster than its handlers** — the warning arrives while there is still time to act.
- The bird isn't a test of the mine in general. It's a small, fast, *expendable* sample of the same air everyone else is breathing.
- Two things make it work: the bird is in the **real** mine, not a model of one, and there are miners breathing the same air right now to compare it against.
- Take either away and you have something else: a laboratory experiment, or yesterday's air quality reading.

*Canarying is “a partial and time-limited deployment of a change in a service and its evaluation.” The part receiving the change is **the canary**; the remainder is **the control**. Canarying is effectively an A/B testing process.*

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Canary / control	The subset receiving the change, and the remainder that doesn't.	5% of production on the release candidate; 95% on the live version.	The canary is usually a much smaller subset of production, or affects a much smaller subset of users. The fraction of total load you run on the canary should be proportional to the size of the canary population — otherwise you're testing a different system, not a smaller one.
Before/after evaluation	Replace the old system entirely, then compare behaviour before and after the change.	A blue/green cutover evaluated by comparing this week to last.	One might call it “ a canary deployment in time-space ” — you segment by time rather than by population. Risky because time is one of the biggest sources of change in observed metrics : release on Monday and you may be comparing a business day to a weekend, with users behaving completely differently. The degradation you see may have happened to the control too.

Gradual canary	Multiple stages of increasing population, rather than one evaluation.	A small first stage, then larger ones once it passes.	The point isn't just safety — it's metric selection . In the first stage you have no confidence, so you use only the clearest indications of a problem: application crashes, request failures. Later stages have enough population to make noisier metrics meaningful.
Blue/green deployment	Two instances: green serving traffic, blue ready to serve. Deploy to blue, then move traffic.	Cutover without downtime; rollback is a trivial reversal of the router change.	Costs twice the resources . And used as normally described, it is a before/after canary , with all that implies. It becomes a real A/B canary only if you run both simultaneously and split traffic — green as control, blue as canary, population controlled by the traffic split.
Artificial load generation	Driving the candidate with synthetic traffic instead of real users.	Load tests in QA, preproduction, or even production.	Not canarying by definition, though still useful. Good at code coverage , poor at state coverage — especially hard in mutable systems with caches, cookies and request affinity — and it may not model organic traffic shifts. Also genuinely dangerous in some systems: synthetic load on a billing system might send callouts to card providers and actually charge customers .
Traffic teeing	Copy real traffic to both systems; production serves and responds, the canary serves the copy and discards the responses.	You can even compare canary responses to real ones for further analysis.	Representative traffic, but more complicated to set up , and it doesn't identify risk in stateful systems: if the canary and production share a cache, an artificially inflated hit rate invalidates your performance measurements .

Release engineering, the foundation underneath

- Five principles: **reproducible builds** (same inputs, same artifacts, this week as last); **automated builds** (check in code, automation produces and uploads artifacts); **automated tests**; **automated deployments** (performed by computers, not humans); **small deployments** (artifacts contain small, self-contained changes).
- What they buy: less operational load; enforced peer review and version control, since automation is code; consistent repeatable processes with fewer mistakes; and a pipeline you can **monitor and measure** — how long does a release take to reach production, how often are releases successful, what would catch defects earlier, what can be parallelised.
- The virtuous cycle: automation lets you release more often → fewer changes bundled per artifact → **cheaper and easier rollback** → and bug fixes reach users faster.
- A related lever: **separate components that change at different rates** — binaries, environments like the JVM, kernel and OS, libraries, service config and flags, feature and experiment config, user config. Feature flag frameworks let you separate feature launches from binary releases, so a binary carrying several features can enable them one at a time, and **selectively disable just the misbehaving ones** without waiting for the next build cycle.

The arithmetic that justifies the whole practice

- Take a release candidate that fails **20% of requests**. Deployed globally, you fail 20% of everything, immediately.
- Deploy it to a **5% canary population** instead: 20% errors on 5% of traffic gives a **1% overall error rate**. Same defect, same detection and rollback time — a twentieth of the damage.
- The general statement: **impact on the budget is directly proportional to the amount of traffic exposed to defects**. That is the sentence to remember.
- But notice what the arithmetic hides — the errors become **hard to see**. At 1% overall the canary's failure may be indistinguishable from background noise, which is why metric breakdown by version matters (below).

THE MODEL'S ASSUMPTIONS, STATED HONESTLY

That calculation assumes **uniform load**; that you may spend your *entire* error budget on canaries; that only unavailability introduced by new releases counts, rather than actual availability; and a **100% failure rate as the worst case**, when real defects rarely affect all usage. It also allows overall availability to dip below the objective for the canary's duration.

“Essentially, all models are wrong, but some are useful.” Use the **simplest model that meets your technical and business objectives** — chasing technical correctness here leads to overinvestment in modelling, and for complex services to incessant tuning for no real benefit.

Choosing population and duration

- **Development velocity sets the ceiling**. Release weekly and you can afford fairly long canaries. Release daily and a week-long canary is impossible. Deploy continuously — twenty times a day — and the duration must be significantly shorter.
- **Run one canary at a time**. Simultaneous canaries add significant mental effort to track system state, which becomes a real problem in exactly the nonstandard circumstances where reasoning quickly matters — and overlapping canaries risk **signal contamination**. The advice is strong: run only one.
- Four dimensions to balance, and they can be **mutually at odds**:

DIMENSION	WHAT IT DEMANDS
Size and duration	Big enough and long enough to be representative. Terminating after a handful of queries gives no useful signal for systems with diverse queries. The higher the processing rate, the less time you need to be confident the behaviour is attributable to the change rather than a random artifact.
Traffic volume	Enough traffic that the system has handled a representative sample <i>and</i> had a chance to react badly to it. The more homogeneous the requests, the less volume you need.
Time of day	Performance defects typically manifest only under heavy load — resource contention, database write conflicts, locking in multithreaded applications. Deploying off-peak likely won't trigger them.
Metrics to evaluate	Trivial metrics like query success evaluate quickly; something like queue depth may need more time or a larger population to give a clear signal.

Once you have metrics from past canaries, you can choose parameters based on **typical canary evaluation failure rates** rather than hypothetical worst cases.

Choosing metrics — where most canary processes go wrong

- **The metric must indicate problems.** Start from your indicators: good ones already have strong attribution to service health, and you've done the work. A failed user request is probably a problem — though be disciplined, since **a request can fail because the user asked for something unreasonable**, like a nonexistent URL.
- **More metrics is not better.** Each needs a defined notion of acceptable behaviour. Too strict and you get false positives — you'll believe a good canary is bad. Too loose and bad canaries go undetected. Getting it right is time-consuming and needs analysis, and expectations must be re-evaluated as the service evolves.
- So **stack-rank metrics by how well they indicate user-perceivable problems** and take the top few — **perhaps no more than a dozen**. Beyond that, returns diminish and are outweighed by maintenance cost, or by the damage to trust in the release process when they aren't maintained.
- For the example frontend service: candidate metrics include CPU usage, memory footprint, HTTP return codes, latency, correctness. The best are **HTTP return codes and latency**, because their degradation maps most closely to a problem that impacts users. **CPU usage is a poor choice** — increased resource usage doesn't necessarily impact the service, and produces a flaky, noisy canary process, which leads to the canary being **disabled or ignored by operators**, defeating the point of having one.
- The tricky case: **HTTP 404**. It can mean the user followed a wrong URL — imagine a broken link shared on a popular discussion board — or that the server incorrectly stopped serving a resource. The workaround is to **exclude 400-level codes from canary evaluation** and add black-box monitoring testing for the presence of a particular URL, then feed that into the analysis. This isolates the canary process from odd user behaviour.

REPRESENTATIVE AND ATTRIBUTABLE — THE ISOLATION PROBLEM

- In a large population you will have **outliers**: oversubscribed machines, machines on different kernels with different performance characteristics, machines on an overloaded network segment. So **the difference between canary and control is as much a function of the two infrastructures as of the change you deployed**.
- **Isolate the metric itself.** Whole-system CPU is poor — database load and log rotation move it too. Better: **CPU time spent by the process serving the request**. Better still: **CPU time serving the request over the window the process was actually scheduled on a CPU**. An oversubscribed neighbour is a real problem and monitoring should catch it, but it isn't caused by your change and shouldn't fail your canary.
- **Tie the metric back to indicators.** If a metric can change wildly with no impact on the service, it is unlikely to make a good canary metric.
- **Shared failure domains cut both ways.** A bad canary can negatively impact the control; bad system behaviour can make you misjudge the canary. Two consecutive requests from one client can cross the boundary — the canary's response changes the content of the second request, which lands on the control and alters *its* behaviour. So a stop signal **does not necessarily mean the canary is at fault**, and you should also use **absolute measures, such as defined objectives**, to confirm the system is operating correctly.

What monitoring has to be able to do

- **Fine-grained breakdown.** A 5% canary at a 20% error rate shows up as 1% overall — potentially indistinguishable from other sources of error. Broken down **by population servicing the request**, the same data shows the release candidate serving the vast majority of errors while the live version sits at a low steady state. Monitoring that reasons well about a whole service **is not sufficient** to analyse a canary.
- **Matching intervals.** Canaries are time-limited by design, so an aggregated metric can wreck the evaluation. The failure runs: an unrelated event causes some errors; a canary deploys to 5% for 30 minutes; the system evaluates the *errors-per-hour* metric; the deployment is judged bad because errors-per-hour differs from the control. The metric was computed over an hour to judge a 30-minute deployment. **Make metric intervals the same as, or less than, the canary duration.**

Canarying things that aren't request/response

- **Duration follows the work unit.** For interactive systems we assume processing takes seconds, far shorter than the canary. For rendering pipelines or video encoding it can take much longer, so the canary duration must **at minimum span a single work unit**.
- **Isolation gets harder.** Many pipelines have one work assigner and a fleet of workers; in multistage pipelines a unit is processed, returned to the pool, then picked up again. Ensure the workers processing a particular unit always come from **the same pool** — canary or control — or the signals get increasingly mixed.
- **Metrics get harder.** You may care about end-to-end time to process the unit (the analogue of latency) as well as the quality of the processing itself, which is entirely application-specific.

CONCEPT BOUNDARY — WHAT MAKES SOMETHING A CANARY

It is not simply “a careful rollout”, and it is not any comparison that produces a graph.

It is a *partial* and *time-limited* deployment evaluated against a **concurrent control** carrying **real traffic**. Remove the concurrency and you have before/after evaluation, where time itself is the biggest confounder. Remove the real traffic and you have load testing, which gives good code coverage and poor state coverage. Both are useful; neither tells you what a canary tells you.

Anchor sketch — the canary decision chain

Legend: boxed = a named node **reversed = focus node** Q = self-test cue

a release candidate

|

v

POPULATION -- 5% of traffic, load proportional to population

|

impact on budget is proportional to

|

the traffic exposed to defects

v

DURATION -- bounded above by release cadence

|

bounded below by: representative sample,

|

traffic volume, time of day, metric type

|

ONE canary at a time

v

METRICS -- start from your SLIs; stack-rank by

|

user-perceivable impact; no more than a dozen

|

good: HTTP codes, latency

|

bad: whole-system CPU (noisy -> ignored)

v

COMPARISON -- against a CONCURRENT control

|

broken down by version, not service-wide

|

metric interval <= canary duration

v

good / bad -> proceed, or pause and roll back

Q Why is whole-system CPU a bad canary metric, and what is the failure mode of choosing it — not for the metric, but for the process?

Q Give the mechanism by which a metric's *time interval* can ruin an otherwise sound canary, and the rule that prevents it.

Q Give the sentence that connects canary population size to the error budget.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“We'll roll it out to a few servers first.”	“A partial, time-limited deployment evaluated against a concurrent control — with the load fraction proportional to the canary population.”
“It only affects a few users if it's broken.”	“Impact on the error budget is directly proportional to the traffic exposed to defects — 20% errors on a 5% canary is a 1% overall rate.”
“We compared it to last week.”	“That's a before/after evaluation — a canary in time-space. Time is one of the biggest sources of change in observed metrics, so a Monday release compared to a weekend is mostly noise.”
“We watch CPU during the rollout.”	“Weak canary metric — resource usage doesn't necessarily impact users, and a noisy canary gets disabled or ignored, which defeats the process.”
“The dashboard didn't show anything.”	“Service-wide monitoring isn't sufficient. You need fine-grained breakdown by the population servicing the request, or a 5% canary disappears into the aggregate.”
“The canary looked bad so we reverted.”	“Right call — but note that a stop signal doesn't prove the canary is at fault. Shared failure domains mean the control can be affected too, so confirm against absolute measures like the objective.”
“We load-test in preproduction instead.”	“Good code coverage, poor state coverage — synthetic load struggles with caches, cookies and request affinity, and won't model organic traffic shifts.”
“We use blue/green, so we're covered.”	“Used as a cutover that's a before/after canary. Run blue and green simultaneously and split traffic and it becomes a real A/B canary.”

PART 4 · QUESTION 1

A team deploys twenty times a day. Their canary process runs each candidate at 2% of traffic for six hours, evaluating on error rate, p99 latency, memory footprint, host CPU, thread count, cache hit ratio, GC pause time, and open file descriptors. Canaries frequently fail; engineers have started re-running them until they pass.

(a) Name the two parameter choices that are inconsistent with a twenty-a-day cadence, and say what each one breaks. **(b)** Diagnose the re-running behaviour: name the underlying metric problem, and explain why it is more dangerous than a canary that simply misses defects. **(c)** Give the selection principle you'd apply, and name which of their eight metrics you would keep.

PART 4 · QUESTION 2

A video transcoding pipeline canaries a new encoder. The team deploys the candidate to 10% of workers, watches for twenty minutes, sees no change in the service-wide error rate, and promotes it. Two days later, complaints arrive about corrupted output on long files, and the metric charts from the canary window show nothing unusual.

(a) Name the two errors specific to canarying a non-interactive system, and say what each one made invisible. **(b)** Explain why the service-wide error rate was the wrong signal, using the 5%/20% arithmetic to make the point. **(c)** Give the isolation requirement peculiar to multistage pipelines and what goes wrong without it, then say why simply enlarging the canary population — the obvious response to “we saw nothing” — would not have helped.

2×2 — is this actually a canary?

COLUMNS: DO REAL USERS RECEIVE THE RESPONSES? →

	Yes the candidate actually serves someone	No responses discarded, or the traffic is synthetic
CONCURRENT control A and B running together	A true canary Partial, time-limited, evaluated against the remainder of production carrying live traffic at the same moment. <i>Move:</i> this is the only cell that answers “is this release safe for my users, now.” Break metrics down by version, keep intervals inside the canary duration, and run one at a time.	Teeing Copy real traffic to the candidate, which serves it and discards the responses. <i>Move:</i> real traffic, but nobody is served by the candidate — so it's safe, harder to set up, and worthless for stateful risk. A shared cache inflates the hit rate and invalidates the performance numbers.
NO concurrent control compared against another time	Before/after The whole system is replaced and you compare to the previous period. A blue/green cutover lands here. <i>Move:</i> accept that time is the confounder — a Monday-versus-weekend comparison is mostly noise. If you must, keep the window short and compare like periods.	Load testing Synthetic load in QA, preproduction, or production. <i>Move:</i> keep doing it — good code coverage — but don't call it a canary. Poor state coverage, and dangerous in mutable systems where synthetic traffic can charge real credit cards.

Read it as: only the top-left cell tells you about *this release, for these users, right now* — a concurrent control, and real users actually receiving the candidate's answers. The other three are worth doing and answer different questions; the mistake is not doing them, it's believing they substitute for the first.

CARRY-OUT RULE FROM THIS PART

Buy information at a price you set. The defect rate is not yours to choose, but the exposure is — and cost to your budget is directly proportional to the traffic you expose. Pick the smallest population that still gives a representative signal, watch the fewest metrics that map to real user harm, and compare against something running at the same moment.

Concept sketch — the whole competency on one page

Three named groups, in the order the story runs: **SET** → **ENFORCE** → **SPEND**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: SET — pick a number below perfect ENFORCE — make it bite SPEND — buy information

reversed = the one node to remember Q = retrieval cue

GROUP 1 - SET . a number below perfect

100% is wrong: unreachable . invisible . blocks change . reactive

```

|
+--> SLI  good events / total events, 0-100%
|         specification (the question) vs implementation (the
|         instrument) -- quality . coverage . cost
+--> SLO  the target, owned by someone who can trade
|         feature velocity against reliability
+--> ERROR BUDGET = 100% - SLO

```

|
v

GROUP 2 - ENFORCE . or it is just a KPI

three parties must sign: product . developers . defenders
refusal by each tells you a different thing

```

|
+--> ERROR BUDGET POLICY  what happens, and who does it
+--> check against reality: do dips match complaints?
|         miss real harm = RECALL problem  -> tighten
|         false alarms   = PRECISION problem -> relax
|         neither works  -> fix the IMPLEMENTATION
+--> decide: size incidents by budget consumed,
|         fund the class that costs the most

```

|
v

GROUP 3 - SPEND . deliberately, not accidentally

CANARY vs **CONTROL**, concurrent, on real traffic

```

20% errors on a 5% canary = 1% overall
budget impact is proportional to traffic exposed
|
+--> population . duration . ONE at a time
+--> metrics from your SLIs, ranked, at most a dozen
+--> break down BY VERSION; interval <= duration
+--> not a canary: before/after . load test . teeing

```

The question per node — cover the right column

GROUP 1 — SET

NODE	QUESTION	ANSWER IN ONE LINE
Why not 100%	Give all four reasons.	You can't reach it even with redundancy and failover; users can't see it through a long dependency chain, so each nine costs more for near-zero marginal utility; you could never change, and change is the number one source of outages; and it leaves you purely reactive.
SLI shape	What form, and why that form?	Good events over total events. It lands 0–100%, makes the budget simply 100% minus the objective, and lets every tool expect numerator, denominator and threshold.
Specification vs implementation	What's the difference, and where do the compromises live?	The specification is the outcome that matters to users; the implementation is how you measure it. All the blind spots live in the implementation — logs miss requests that never arrived, probes miss subsets of users.
Quality vs coverage	Why aren't they the same?	Quality is how accurately you capture one customer's experience; coverage is how many customers you speak for. An instrument can be perfect about 2% of users.
Reversed: error budget	Why is this the node to remember?	It converts reliability from an opinion into a currency — spendable, attributable to incidents, comparable across projects. Everything in groups 2 and 3 is about protecting or spending it.
Ownership	Who owns the objective and why them?	Someone empowered to trade feature velocity against reliability — normally the product owner, or the CTO in a small organisation.

GROUP 2 — ENFORCE

NODE	QUESTION	ANSWER IN ONE LINE
The three signatures	Who signs, and what does each refusal mean?	Product managers (refusal means it's too loose), developers (they must commit to acting on exhaustion), and the defending team (refusal means it isn't defensible without excessive toil and burnout).
Error budget policy	What makes it more than paperwork?	It names the actions and the owners, and getting it approved is itself the test of whether the objectives are fit for purpose.
Recall vs precision	Define both, and give the trap.	Recall is the share of user-impacting events you capture; precision is the share of captured events that were user-impacting. The trap: improving recall while destroying precision produces alerts the team learns to ignore.
Aspirational objective	When, and what does it avoid?	When you need a tighter target you can't yet meet. Tracked alongside but explicitly excluded from the policy, so you avoid a permanent state of emergency.
Sizing incidents	What does budget-consumed let you do that severity labels don't?	Compare classes: if bad pushes cost twice what database failures do, fund the release problem rather than the rarer, scarier one.
The independence fallacy	Why doesn't two zones give you six nines?	Because the multiplication assumes independence, and there are common dependencies, common failure domains, shared fate and global control planes.

GROUP 3 — SPEND

NODE	QUESTION	ANSWER IN ONE LINE
------	----------	--------------------

Canary and control	Define both, and the one sizing rule.	The canary is the subset receiving the change, the control the remainder. The fraction of load should be proportional to the size of the canary population.
The 5%/20% arithmetic	Do the sum and state the general rule.	20% errors on a 5% canary is 1% overall. Impact on the budget is directly proportional to the traffic exposed to defects.
One at a time	Why not run several?	Tracking system state gets much harder exactly when you need to reason quickly, and overlapping canaries risk signal contamination.
Metric selection	Where do you start, and how many?	Start from your indicators, stack-rank by user-perceivable impact, keep the top few — perhaps no more than a dozen.
The noisy-metric failure	What actually goes wrong with a bad metric like host CPU?	It makes the canary flaky, and a flaky canary gets disabled or ignored by operators — which defeats the point of having one at all.
Monitoring requirements	Two things monitoring must do.	Break metrics down by the population servicing the request, and use intervals no longer than the canary duration.
Not a canary	Name three near-misses and what each lacks.	Before/after (no concurrent control, time confounds it), load testing (synthetic, poor state coverage), teeing (real traffic but blind to stateful risk through shared caches).

Master 2x2 — the decision card you can carry to other problems

This one isn't about software, which is the point. Two questions decide whether a setback teaches you anything: **did you choose it**, and **could you afford it**.

COLUMNS: COULD YOU AFFORD IT? →

	Within budget you had room, and knew it	Beyond budget no room, or you never measured
FAILURE CHOSEN you picked the size and timing	Buying information A canary at 5%. A fire drill. A practice exam. A small deliberate loss taken while you're watching. <i>Move:</i> this is the only quadrant that reliably produces learning. Spend here on purpose — an unspent budget is not a saving, because something else will spend it for you.	Gambling Deliberately degrading service to see what users tolerate, with no budget to absorb it. <i>Move:</i> don't. Experiment with reliability only when confident you have budget to burn — and treat deliberately worsening the customer experience as a line to cross very thoughtfully, if at all.
FAILURE IMPOSED it happened to you	Absorbed An outage that ate 13% of the budget. Survivable, and it still bought you data — which class of incident, and what it cost. <i>Move:</i> attribute it. An imposed failure you measure is half as wasteful as one you merely survive.	Crisis Budget exhausted by events you didn't choose and can't absorb. Every option now is a reaction. <i>Move:</i> this is what the whole discipline exists to prevent. The policy that fires here — freeze, reprioritise, declare emergency — should have been agreed while everyone was calm, because nobody negotiates well from this cell.

The move that matters: you don't control the right-hand column directly — you control it in advance, by setting a budget honestly and not spending it all on accidents. And you control the row completely: choosing your failures is always available, and always cheaper than having them chosen for you.

THE THUMB RULE, IN ONE SENTENCE

Decide in advance how much failure you can afford, then spend it deliberately to learn something — because if you don't spend it, something else will.

Far outside software. A hospital budgets theatre time for training operations rather than learning on emergencies. An investor sizes a position so a loss is tuition, not ruin. A cyclist rides the descent in the rain on a quiet Sunday rather than discovering their braking limit in a race. In each case the total risk isn't reduced — it's *rescheduled* to a moment of your choosing, at a size you picked.

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Objectives and budgets

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Nobody will say what the reliability target is, but everyone acts as if it's perfection.	An unspoken 100% goal — the second maturity level.	Get agreement that a number below 100% is acceptable, before arguing about which number.
The target is published and charted, and missing it changes nothing.	An objective without teeth. It's a KPI, not a decision-making tool.	Write the error budget policy and get it approved by product, developers and the defending team.
The reliability number is owned by the people who operate the service.	Ownership sits with someone who can't trade feature velocity against reliability.	Move ownership to the product owner, or the CTO in a small organisation.
Dashboard is green; support tickets are climbing.	A recall problem — the indicator isn't capturing user-impacting events.	Tighten the objective, or move the measurement closer to the user. Check the correlation between tickets and budget loss.
The team has stopped believing the alerts.	A precision problem, probably created by chasing recall.	Relax the objective, or fix the implementation. An alert nobody believes is worse than none.
You need a tighter number but can't meet it for months.	Tightening now would mean permanent budget exhaustion.	Institute an aspirational objective — tracked alongside, explicitly excluded from the policy.
Availability measured from server logs looks far better than users report.	The implementation misses requests that never reached the backend.	Move to load balancer monitoring, black-box probes, or client-side instrumentation — and name the new blind spot too.
The latency number looks fine and users say it's slow.	A single threshold hiding the tail.	Add grades: 90% under one threshold and 99% under a looser one.
Your indicator swings for no apparent reason between months.	A calendar or 30-day window containing a varying number of weekends.	Use a rolling window of an integral number of weeks — four is the general-purpose recommendation.
“Two zones gives us six nines.”	The independence fallacy.	Enumerate common dependencies, failure domains, shared fate and global control planes before believing any such multiplication.
Budget blown by another team's outage; yours behaved perfectly.	A dependency-caused miss, with no pre-agreed answer.	Freezing anyway makes users happier. Whatever you choose, record it in the documented policy before it happens again.
Service runs flawlessly, needs almost no attention.	Met objective, low toil, high satisfaction.	Relax release processes and increase velocity, or move to a less hands-on support tier and redirect the attention.

Canarying

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
A bad release reached everyone at once.	No canary — global deployment exposes 100% of traffic to the defect.	Deploy to a small population first; budget impact is proportional to traffic exposed.
The canary is running but you can't see its effect in the graphs.	Service-wide monitoring — a 5% canary at 20% errors reads as 1% overall.	Break metrics down by the population servicing the request.
The canary was judged bad, and nothing was actually wrong.	Metric interval longer than the canary duration, or a metric with high variance.	Make intervals no longer than the canary; drop metrics that can move without user impact.

Engineers re-run canaries until they pass.	Too many metrics, or thresholds too strict — a flaky process losing its authority.	Stack-rank by user-perceivable impact and keep the top few, perhaps no more than a dozen.
The canary looks bad but so does the control.	Imperfect isolation — shared failure domains, or cross-contamination between requests.	Don't assume the canary is at fault. Confirm against absolute measures such as the objective.
Canary passed off-peak; the defect appeared at peak.	Performance defects manifest under load — contention, write conflicts, locking.	Canary at a representative time of day, not just a convenient one.
Canarying a pipeline found nothing, then output was corrupt.	Canary duration shorter than a single work unit.	Span at least one work unit, and keep each unit's processing within one worker pool.
Several canaries in flight, and nobody can say what's deployed where.	Simultaneous canaries — mental load plus signal contamination.	Run one at a time.
“We compared this week to last week.”	Before/after evaluation — a canary in time-space.	Use a concurrent control. If you can't, keep the window short and compare like periods.
Synthetic load in preproduction found nothing; production broke.	Good code coverage, poor state coverage — caches, cookies, request affinity.	Keep the load tests, but don't treat them as a canary substitute.

THE MAPPING IN THE OTHER DIRECTION — WHAT EACH MOVE PROTECTS YOU FROM

MOVE	WHAT IT PROTECTS YOU FROM
An objective below 100%	Unbounded spend on extra nines nobody perceives; the change freeze perfection imposes; a purely reactive posture.
Error budget	Reliability arguments that can't be settled, and an inability to rank either incidents or candidate projects.
Error budget policy	An objective that changes nobody's behaviour — a KPI wearing a decision-maker's clothes.
Good-over-total indicator shape	Bespoke tooling per metric, and a scale nobody can reason about or turn into a budget.
Moving the implementation closer to the user	Confident numbers that speak for the wrong population — availability that can't see requests which never arrived.
Multiple grades on one indicator	A healthy-looking figure hiding an unhappy tail.
Watermark checked by clients	Freshness weighted by records rather than by how many users actually saw stale data.
Rolling window of integral weeks	Indicator movement caused by how many weekends a period contained.
Aspirational objective	Permanent budget exhaustion, and a policy that fires so often it stops being enforced.
Critical user journeys	Per-request success that passes while the user's actual task fails.
Bucketing	One objective averaging premium against free, and interactive against batch.
Canarying at all	A defect reaching 100% of traffic at once.
A small canary population	Budget loss, which is directly proportional to the traffic exposed to the defect.
One canary at a time	Signal contamination, and an unreasonable mental load exactly when you need to reason fast.
Breakdown by version	A 5% canary disappearing into a 1% aggregate that looks like ordinary noise.
Metric interval no longer than the canary	Pre-canary contamination condemning a perfectly good release.
A concurrent control	Time itself confounding the comparison — Monday judged against a weekend.

Numbers worth remembering

NUMBER	WHAT IT IS	WHY IT MATTERS IN AN ARGUMENT
3,000 of 3,000,000	The error budget from a 99.9% objective over four weeks. A 1,500-error outage costs half of it.	The cleanest demonstration that a budget is a countable quantity you can attribute to a specific incident.
97.123% → 97%	Measured availability rounded to two significant figures to get a starting objective.	Shows the whole method: measure, round down, publish. Your first objective is derived, not debated.
432 ms / 891 ms	Measured 90th and 99th percentile latency, becoming objectives of 450 ms and 900 ms.	Two grades on one indicator — the concrete answer to “the average looks fine.”
50 ms	The rounding granularity for latency objectives.	Because users are unlikely to perceive a 50 ms change — though the right window differs between a reporting service and a real-time game.
109,897	Allowed failures under a 97% objective over four weeks.	The denominator that turns “that was a bad outage” into “that was 13% of the quarter’s budget.”
13% and 65%	Budget consumed by a four-hour bad release (14,066 errors) and a 20-hour database restore (72,000 errors).	The pair that lets you rank incidents by cost rather than by how frightening they felt.
2×	Bad pushes cost roughly twice the error budget of database failures — two or three per year against one in five years.	The argument-ending comparison: fund the release problem, not the rarer, scarier one.
Four weeks, rolling	The general-purpose window, plus weekly summaries for tasks and quarterly for projects.	An integral number of weeks, so weekend traffic doesn't move the number for uninteresting reasons.
Five or fewer	Indicator types to choose for a service.	Stops the “measure everything” instinct that produces a dashboard nobody reads.
20% on 5% = 1%	Canary arithmetic: a candidate failing 20% of requests, deployed to 5% of traffic.	The single most useful number here — and it generalises: budget impact is directly proportional to traffic exposed.
No more than a dozen	Metrics to evaluate in a canary process.	Past that, returns are outweighed by maintenance cost and by lost trust when they aren't maintained.
One	Canary deployments in flight at any moment.	Simultaneous canaries add mental load exactly when reasoning must be fast, and risk signal contamination.
99.99% / 99.9%	Bucketed availability objectives for premium and free customer tiers.	Concrete proof that one objective for all requests is a choice, not a law.

Glossary — the terms you should be able to define cold

TERM	DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR
SLI / SLO / SLA	Indicator = what you measure. Objective = the target you hold it to. Agreement = a business contract with compensation when users are unhappy enough to be owed something.
Error budget	100% minus the objective, in the indicator's own units. A budget only if someone may act on its exhaustion.
Error budget policy	The written actions and owners on exhaustion. Its approval is the test of whether the objective is fit for purpose.
SLI specification	The user-relevant outcome, independent of measurement. Distinct from the implementation, which is the instrument and carries the blind spots.
Quality / coverage / cost	How accurately one customer's experience is captured; how many customers are represented; what it costs to build. Quality and coverage are independent.

Request-driven / pipeline / storage	The three component types, with availability-latency-quality, freshness-correctness-coverage, and durability as their respective indicators.
Watermark	A timestamp the pipeline records for when data was updated — the basis of a freshness indicator, best checked by clients on read so it's weighted by who saw stale data.
Recall / precision	Share of user-impacting events captured; share of captured events that were user-impacting. They trade off, and destroying precision to chase recall produces ignored alerts.
Aspirational objective	A tighter target tracked alongside the live one, explicitly excluded from the policy so you avoid permanent emergency.
Critical user journey	A core multi-step task, like search → cart → purchase. Improves recall without affecting precision; hard to infer from logs.
Bucketing	Labelling indicators and applying different objectives per label — by customer tier, or by expected responsiveness.
Critical dependency	One whose unavailability makes your service unavailable. Its guarantee must be at least as high as the action depending on it.
Canary / control	The subset receiving the change and the remainder that doesn't, evaluated concurrently on real traffic. Load fraction proportional to population size.
Before/after evaluation	A canary in time-space. Time is the confounder — a Monday release compared to a weekend is mostly noise.
Gradual canary	Staged populations, so early stages use only the clearest problem signals and later ones can afford noisier metrics.
Blue/green	Two environments, one serving. A before/after canary unless you run both and split traffic, which makes it a true A/B.
Teeing	Copying real traffic to a candidate that discards its responses. Representative, but blind to stateful risk through shared caches.
Attribution	The property that a metric change is caused by the change you deployed, not by infrastructure differences between canary and control populations.

Answer key

PART 1 · Q1 — THE PROUD 100% TEAM

(a) Three costs. **Quarterly releases** — holding perfection means you can never change, and change is the number one source of outages, so the promise is being paid for in stagnation. **An exhausted rota** — at 100% you can only ever react to the sub-100% that is guaranteed to happen, which is an operations posture rather than an engineering one. **Unbounded spend on reliability** — each additional nine costs more while the marginal utility to customers approaches zero, and users can't perceive it anyway through the chain of systems between you and them.

(b) Because the record is evidence for the wrong proposition. Eleven minutes of downtime proves the target is *nearly* achievable, which makes it feel like a working policy rather than a cost. Nobody sees the counterfactual — the features not shipped, the engineers not sleeping, the improvements not attempted. Success at a bad target is the hardest kind of argument to win, because the damage is entirely in what didn't happen.

(c) Not the number — **ownership**. The objective must be owned by someone empowered to trade feature velocity against reliability; in a small organisation the CTO, more usually the product owner or product manager. Until someone with that authority owns it, any number you pick will lose every argument it has with a launch date. Credit also for naming the accompanying precondition: agreement that a target below 100% is acceptable at all.

PART 1 · Q2 — THE 99.5% THAT NOTHING FOLLOWS FROM

(a) They have an **objective without teeth** — the third maturity level. What's missing is a committed error budget policy, so the objective functions as a KPI or reporting metric rather than a decision-making tool.

(b) Precondition 3 is plainly unmet: the organisation has not committed to using the budget for decision making and prioritising, formalised in a policy. Precondition 1 is almost certainly unmet too — a number published on a wiki is not evidence that all stakeholders *approved* it as fit for the product. Precondition 4, a process for refining the objective, shows no sign of existing either. Any single failure is sufficient because the preconditions are conjunctive: without all four you cannot adopt an error-budget-based approach, and compliance degrades to reporting.

(c) An **error budget policy**, approved by the product manager, the development team and the team defending the service. It must specify at least: the actions taken when the budget is exhausted and *who* takes them; the authors, reviewers and approvers with approval and next-review dates; and a clear escalation path if there is disagreement about the calculation or whether the actions are appropriate. The near-miss fix — publishing the number more prominently — changes nothing, because visibility was never the constraint. Nobody was empowered to act, and a more visible number that still causes nothing to happen is just a better-lit dashboard.

PART 2 · Q1 — 99.95% FROM THE LOGS, COMPLAINTS FROM ONE CARRIER

(a) The **specification** — the proportion of user requests that succeed — is probably right; the **implementation** is the problem. Server logs can only record requests that arrived, so requests that never reached the backend are invisible. A carrier-specific network problem produces exactly this signature: users fail, the log-derived number stays beautiful, and the two facts are not in conflict because they are measuring different populations.

(b) **Black-box probers** executing JavaScript in a browser in a VM: these catch errors when requests can't reach your network, but may miss issues affecting only a subset of users — and a prober fleet that doesn't sit on that carrier will miss this one too. **Client-side instrumentation** in the page's own JavaScript reporting to a telemetry service: this captures the real user experience most accurately, including failures that never reach you. Load balancer monitoring is worth naming as the cheap intermediate — closer to the user than app logs, often already available — though it still won't see a request that never arrived.

(c) Adopt **load balancer monitoring** first if you have it, because it is cheap and strictly better than logs, then work toward client-side instrumentation. The extra obligation of the most accurate option is that **the telemetry infrastructure has its own reliability requirements** — you now depend on a new system to know whether your system is working, and you must decide what you believe when it fails.

PART 2 · Q2 — THE NIGHTLY RECOMMENDATIONS JOB

(a) It's a **pipeline**. Its three indicator types are **freshness, correctness and coverage**. What the team implemented is none of them cleanly — “runs that completed without error” is closest to an availability measure of the job itself, and it tells you the process exited zero, not that users got good data.

(b) “Days out of date” is **freshness**: have the pipeline record a **watermark** with the timestamp of the last update, and have clients check it when they request data, incrementing one counter for the request and another when the data was fresher than the threshold. That weights the measure by who actually saw stale data. “Plain wrong” is **correctness**: either inject data with known outputs and count how often the output matches, or compute correct output by a separate, more expensive method and sample input/output pairs.

(c) Coverage is measured by exporting the number of records the pipeline *should* have processed against the number it did — and its retained blind spot is that it **cannot count records it never knew about**, typically through misconfiguration. The standing caveat on correctness by curated test data is that the curated data must be **representative of real-world data**; if it isn't, the indicator is representative of nothing, however green it looks. And the near-miss fix fails for a structural reason: **tightening the existing objective changes nothing**, because “runs completed without error” measures the job's exit status, not the data. It has been passing at 100% for a month while both complaints were true, so no threshold on that indicator can ever surface staleness or wrongness — you need different indicators, not a stricter one.

PART 3 · Q1 — CHECKOUT MEETS ITS NUMBER AND FEELS FLAKY

(a) A **recall** problem: significantly user-impacting events are occurring and the indicator isn't capturing them. Precision is not the issue — nothing is alerting falsely; the instrument is silent when it should speak.

(b) Because per-request availability measures individual HTTP requests, and a multi-step flow can fail while every constituent request succeeds — the payment step can return a well-formed response that doesn't complete a purchase, or the user can be dropped between steps. Aggregate per-request success is a poor proxy for “did the user achieve their goal.” The concept that addresses it is the **critical user journey**: a sequence of tasks that is a core part of the user's experience — for shopping, searching for a product, adding to a cart, completing a purchase.

(c) The attractive property is that critical user journeys **improve recall without affecting precision** — you catch more real user harm without generating more false alarms, which is exactly the trade you normally can't get. The practical difficulty is measurement: these tasks almost certainly won't map to existing indicators, each involves multiple complex steps that can fail anywhere, and inferring success or failure from logs is extremely difficult — you cannot easily distinguish a user who failed at step three from one who simply got distracted and left. You'd measure it by joining distinct log events, advanced JavaScript probing, or client-side instrumentation. The near-miss fix — **tightening the per-request availability objective** — cannot work here: every individual request in the flow already succeeded, so an indicator sitting at 100% for these journeys will still sit at 100% however strict the target. The failure happens *between* requests, and no threshold on a per-request measure reaches it.

PART 3 · Q2 — SOMEONE ELSE'S OUTAGE, YOUR BUDGET

(a) Two schools of thought, and both are honest. **One**: your team shouldn't halt releases or divert to reliability work, because your system didn't cause the issue — punishing you for it is arbitrary and demoralising. **Two**: enact the freeze regardless of cause, to minimise the chance of further outages while the service is already degraded. The guidance favours the second, on a single ground: **it will make your users happier**. Users experienced an unreliable service; they neither know nor care which team owns the failing component, and stacking your own risk on top of a bad month is what they'd least want. Flexibility is allowed — depending on the nature of the outage and dependency, freezing may not be practical.

(b) This belongs in the **documented error budget policy**, agreed in advance and recorded for posterity. Deciding now is the wrong time because the question has become a negotiation between people with immediate stakes in the answer — the development lead is arguing a position, not a principle. The general rule holds here: if a service exhausts its budget and stakeholders don't agree the policy should apply, you don't argue it in the moment, you return to the policy approval stage.

(c) Structurally: the identity service is a **critical dependency** — its unavailability makes yours unavailable — so its reliability guarantee must be **at least as high** as the guarantee of the action depending on it, and the team running it must own that objective the way you own yours. If it can't reach that level, engineer around it: use a different component, or add defences. Two named defences: **caching** (so a short outage is invisible), and **graceful degradation** (serve a reduced experience rather than failing). Offline store-and-forward processing also earns credit.

PART 4 · Q1 — TWENTY DEPLOYS A DAY, EIGHT METRICS, ENDLESS RE-RUNS

(a) **Duration**: six hours is incompatible with twenty deployments a day. Canary duration must factor in development velocity, and with continuous deployment it has to be significantly shorter — otherwise canaries queue, or you're forced into simultaneous canaries, which add real mental load and risk signal contamination. Only one canary should be in flight at a time. **Population**: 2% is small enough that slower or noisier metrics may never produce a clear signal in the time available; representativeness depends on the canary being sizeable enough and running long enough, and they have squeezed both.

(b) The re-running is a **false positive** problem: with eight metrics, several of which can move without any user impact — host CPU, memory footprint, thread count, cache hit ratio, file descriptors — the acceptable-behaviour definitions are collectively too strict, and canaries fail for reasons unrelated to the change. It is more dangerous than a canary that misses defects because it destroys the **authority of the process**: a flaky canary gets disabled or ignored by operators, which defeats the point of having one. A canary with poor recall still stops some bad releases; a canary nobody believes stops none, and has consumed the budget for building one.

(c) Start from your **service level indicators** — good ones already have strong attribution to service health, and the work is done — then **stack-rank metrics by how well they indicate actual user-perceivable problems** and keep only the top few, perhaps no more than a dozen but here far fewer. Keep **error rate (HTTP return codes)** and **p99 latency**: degradation in either maps closely to real user harm. Drop host CPU, memory, threads, cache hit ratio and file descriptors as primary signals — an increase in resource usage doesn't necessarily impact the service. Credit for noting that whole-host CPU is doubly bad because it isn't isolated: other processes move it, so a better metric would be CPU time spent by the process serving the request.

PART 4 · Q2 — THE TRANSCODING PIPELINE THAT PASSED

(a) **Duration shorter than a work unit:** twenty minutes cannot evaluate an encoder whose long-file work units take longer than that, so the failures that mattered had not finished — let alone been measured — when the canary was promoted. Canary duration must at minimum span the duration of a single work unit. **Metric selection wrong for the system:** they watched an error rate, but the relevant signals for a pipeline are **end-to-end time to process the work unit** and the **quality of the processing**, which is application-specific — corrupted output is a correctness failure that an error rate will never show, because the job succeeded.

(b) Because service-wide aggregation dilutes the canary until it disappears. The standard illustration: a canary at 5% of the population running at a 20% error rate produces an overall error rate of just 1%, which may be indistinguishable from other sources of error. Here at 10% the dilution is smaller but the principle identical — and since the defect produced *no* errors at all, the aggregate was doubly useless. You must be able to perform **fine-grained breakdowns that differentiate metrics between the canary and control populations**. Monitoring that reasons well about an entire service is not sufficient to analyse a canary.

(c) In multistage pipelines a work unit is processed by a worker, returned to the pool, then picked up for the next stage — possibly by a different worker. The requirement is that the workers processing a particular unit are **always pulled from the same pool**, canary or control. Without it the signals get increasingly mixed: a unit partly processed by each population produces a result attributable to neither, so you cannot tell whether the candidate caused what you're seeing. And the obvious response to “we saw nothing” — **enlarging the canary population** — would not have helped, because neither cause was a sample-size problem. The duration was shorter than a work unit, so the relevant work hadn't finished no matter how many workers ran it; and the metric watched cannot see output that is corrupt but successful, at any population. More workers would have produced more unmeasured corruption, which is not a signal.

HOW TO SCORE YOURSELF

Two marks, both quick. **Completeness:** did you name every element, and — for this competency especially — did you say *who has to agree* and *what it costs*? Answers that describe a mechanism but skip the negotiation are half answers here. **Phrasing:** could you have said it in a review without hand-waving? If you wrote “the monitoring is wrong” where the key says “the specification is right but the implementation can't see requests that never reached the backend,” the idea is there and the mechanism isn't — and the mechanism is what changes anyone's mind.

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 • The four reasons 100% is the wrong target						
2 • SLI / SLO / SLA — and who owns the objective						
3 • Error budget: definition and the 3,000-of-3M example						
4 • The four preconditions, and what fails without them						
5 • Good-over-total shape, and the two properties it buys						
6 • Specification vs implementation; quality vs coverage vs cost						
7 • The three component types and their indicators						
8 • Freshness via watermark; coverage and correctness blind spots						
9 • Rounding measurements into a starter objective						
10 • Rolling vs calendar windows; why integral weeks						
11 • The three signatures and what each refusal means						
12 • Error budget policy — required contents						
13 • Recall vs precision, and the trap between them						
14 • The four moves when the objective lacks coverage						
15 • Sizing incidents by budget; comparing classes not instances						
16 • Critical user journeys; bucketing; the independence fallacy						
17 • The decision matrix — what toil does and doesn't decide						
18 • Canary and control; the 20%-on-5% arithmetic						
19 • Population and duration — the four dimensions						

20 • Canary metric selection, and the noisy-metric failure mode						
21 • Isolation, attribution, and shared failure domains						
22 • Monitoring requirements: breakdown and interval						
23 • Before/after, blue/green, load testing, teeing						
24 • Master matrix — chosen vs imposed, affordable vs not						

Final quiz — no notes, fifteen minutes, write the answers

1. Give all four reasons 100% is the wrong reliability target, then sort them: one is about your own system, one about the chain between you and the user, and two about how your organisation ends up behaving. Which is which?
2. Define an error budget, then compute one: a 99.9% success-ratio objective on a service taking 3 million requests over four weeks. What fraction does a single 1,500-error outage consume?
3. Explain the difference between an SLI specification and an SLI implementation, and give two implementations of the same specification with the blind spot of each.
4. Name the three component types, their indicator types, and the trap hidden in a durability number.
5. Your dashboard is green and support tickets are climbing. Name the problem in the recall/precision vocabulary, and give the two directions in which you can improve an implementation.
6. Who must sign off on an error budget policy, and what does a refusal from each party tell you about the objective?
7. You need a tighter objective but can't meet it for six months. What do you do, and what must the policy say about it?
8. A release candidate fails 20% of requests. Deployed to a 5% canary, what is the overall error rate — and state the general principle in one sentence.
9. Name three things that resemble canarying but aren't, and say what each one lacks.
10. Walk through the timeline by which an aggregated metric wrongly condemns a good canary, and give the one-line rule that prevents it.